Recitation 5

Reinforcement Learning and Learning Theory

10-701: Introduction to Machine Learning

November 13, 2026

1 Introduction to Reinforcement Learning

Recall that a Markov decision process is a tuple $(\mathcal{S}, \mathcal{A}, T, R, \gamma, s_0)$, where:

- $\mathcal{S}$ is the set of states
- $\mathcal{A}$ is the set of actions
- $p : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to [0, 1]$ is the transition function
- $R : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to \mathbb{R}$ is the reward function
- $\gamma \in [0, 1)$ is the discount factor
- s_0 is the start state

A **policy** is any function $\pi : \mathcal{S} \to \mathcal{A}$ mapping from the states to the actions. We say that we are executing some policy π if, whenever we are in state s, we take action $a = \pi(s)$.

For the optimal policy function π^* we can compute its **value function** at state s as:

$$\begin{aligned} V^{\pi^*}(s) &= V^*(s) \\ &= \mathbb{E}\left[R(s_0, \pi^*(s_0), s_1) + \gamma R(s_1, \pi^*(s_1), s_2) + \gamma^2 R(s_2, \pi^*(s_2), s_3) \cdots \mid s_0 = s, \pi^*\right]. \end{aligned}$$

In other words, this is the best possible expected total payoff that can be attained using **any policy** π.

This **optimal value function** can be represented using Bellman's equations (named **Bellman optimality equation** for state value function), i.e.,

$$V^*(s) = \max_{a \in \mathcal{A}} \sum_{s' \in \mathcal{S}} p(s'|s, a)(R(s, a, s') + \gamma V^*(s')).$$

If $R(s, a, s') = R(s, a)$ (deterministic reward), then we have the form we saw in class:

$$V^*(s) = \max_{a \in \mathcal{A}} \left\{ R(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a) V^*(s') \right\}.$$

The interpretation of Bellman equation is also very intuitive: "how valuable is the current state under a policy?" Then it naturally follows that the optimal policy $\pi^* : \mathcal{S} \to \mathcal{A}$ can be found as

$$\pi^*(s) = \arg\max_{a \in \mathcal{A}} \sum_{s' \in \mathcal{S}} p(s'|s, a)(R(s, a, s') + \gamma V^*(s')).$$

2 Value Iteration

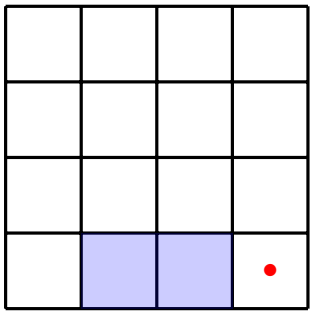


Figure 1: The cliff-walking environment

Here, we assume a 4×4 grid environment. The reward is 1 for reaching the cell with the red dot, -1 for reaching the shaded cells, and 0 for all other cells. The episode ends if the agent lands in either the shaded cells or the red-dot cell. The state space ($\mathcal{S}$) is all the cells. The action space (A) is up, down, left or right. If the agent tries to move out of the grid, it simply goes back to its previous state. The discount factor, γ, is 0.9.

Bellman optimality equation for state value function:

$$V^*(s) = \max_{a \in \mathcal{A}} \sum_{s' \in \mathcal{S}} p(s'|s,a)(R(s,a,s') + \gamma V^*(s'))$$

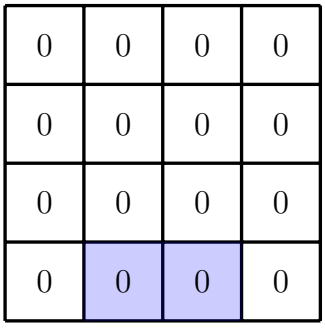
We can numerically approximate V^* using synchronous value iteration and asynchronous value iteration. The recurrence relation is defined as follows for **synchronous** value iteration for all $s \in \mathcal{S}$:

$$V^{(t+1)}(s) = \max_{a \in \mathcal{A}} \sum_{s' \in \mathcal{S}} p(s'|s,a)(R(s,a,s') + \gamma V^{(t)}(s')).$$

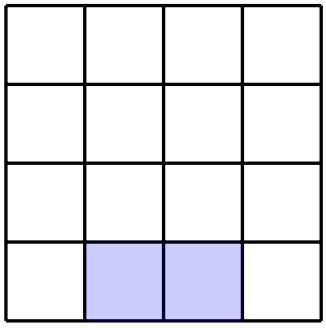
Notice the time term (t), hence the name synchronous. For **asynchronous** value iteration, we use for all $s \in \mathcal{S}$:

$$V(s) = \max_{a \in \mathcal{A}} \sum_{s' \in \mathcal{S}} p(s'|s,a)(R(s,a,s') + \gamma V(s')).$$

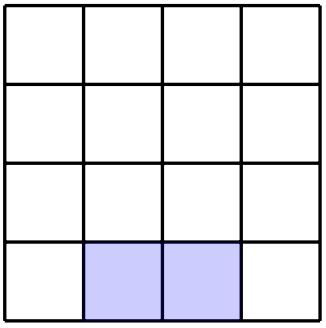
1. Find the updated value of each cell after the first round and after the second round of synchronous value iteration. Assume deterministic transition function.



Initial

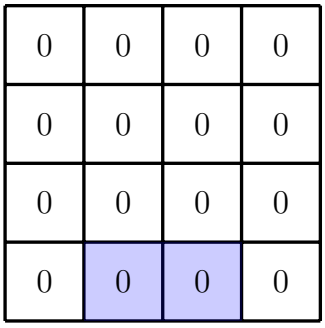


After 1st round

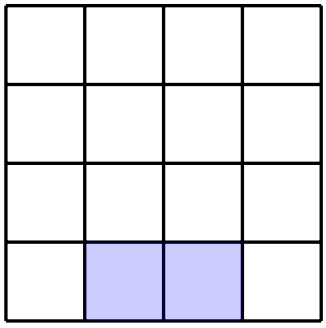


After 2nd round

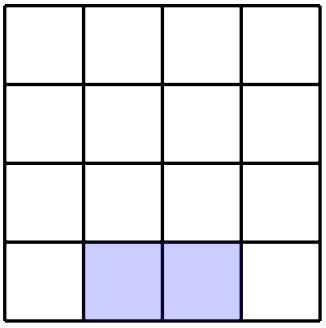
2. Starting over, find the update value of each cell after one round of asynchronous value iteration. Visit each cell in two different orders: (1) bottom right to top left, and (2) top left to bottom right.



Initial

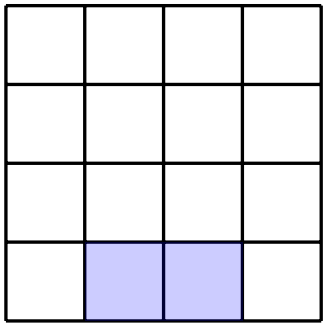


BR to TL

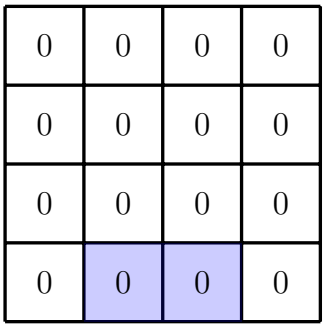


TL to BR

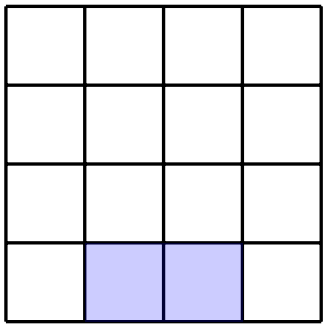
3. What is the policy learned after the one round of asynchronous value iteration when the cells were visited from bottom right to top left? If there is a tie, pick the action that comes first alphabetically (i.e., priority: $\downarrow > \leftarrow > \rightarrow > \uparrow$).



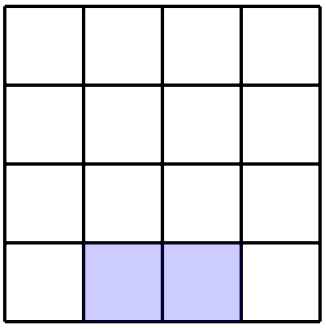
4. Now suppose that the environment is sloped downward towards the cliff, with all the other settings unchanged. For every action taken, there is a 0.5 probability that the agent will move as intended and a 0.5 probability that the agent will slip and move 1 cell down instead. Report the values after two rounds of synchronous value iteration.



Initial



After 1st round



After 2nd round

3 Q-Learning

3.1 Q-Value

We can also define a Q function $Q(s, a)$ that returns the expected discounted future value of taking action a at state s.

Question: When would we want to use $Q(s, a)$ instead of $V(s)$?
Answer:

We can take a part of $V^*(s)$ as $Q^*(s, a)$ so that we have

$$Q^*(s,a) = \sum_{s' \in \mathcal{S}} p(s'|s,a)(R(s,a,s') + \gamma V^*(s'))$$
$$V^*(s) = \max_{a \in \mathcal{A}} \underbrace{\sum_{s' \in \mathcal{S}} p(s'|s,a)(R(s,a,s') + \gamma V^*(s'))}_{Q^*(s,a)} = \max_{a \in \mathcal{A}} Q^*(s,a).$$

With substitution, we have the Bellman Equation for Q-value:

$$Q^*(s,a) = \sum_{s' \in \mathcal{S}} p(s'|s,a)(R(s,a,s') + \gamma \max_{a' \in \mathcal{A}} Q^*(s',a'))$$

Then it follows that the optimal policy can be obtained as

$$\pi^*(s) = \arg\max_{a \in \mathcal{A}} Q^*(s,a).$$

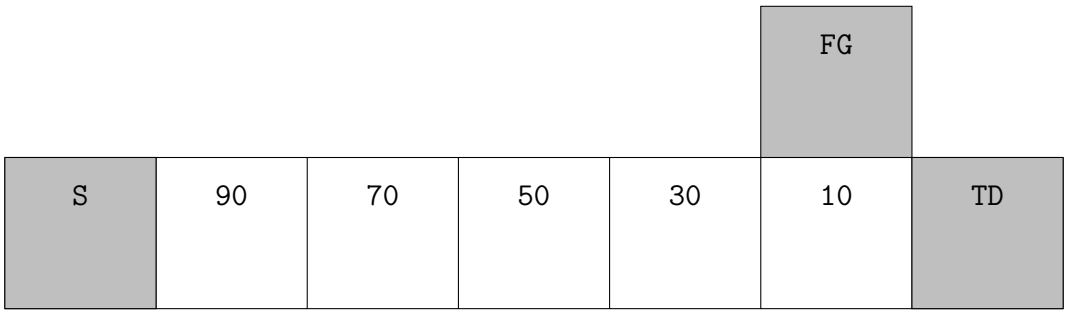
However, *when p and R are unknown*, we cannot compute the expectation in the Bellman Eq. (the summation) directly. Instead, after taking action a in state s and observing reward r and next state s', Q-learning moves $Q(s, a)$ toward a one-sample estimate of this expectation:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \Big[\underbrace{r + \gamma \max_{a' \in \mathcal{A}} Q(s',a')}_{\text{TD target}} - Q(s,a) \Big],$$

where $\alpha \in (0, 1]$ is the learning rate, and the bracketed term is the **temporal difference (TD) error**. If s' is a terminal state, we set $\max_{a'} Q(s', a') = 0$.

3.2 Exercise

Let's play some football (the ~~wrong~~ American kind). We will divide the field into eight states (labeled slightly differently from an official football field for clarity):



Here are some basic rules (slightly different than official football rules, but let's work with it for the sake of this example):

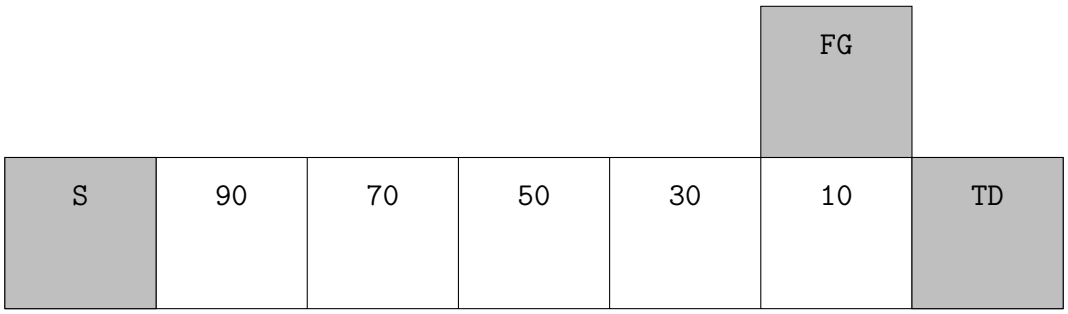
- There are two teams. For simplicity, let's call them Offense and Defense.
- The length of the field is 100 yards; we will set each 20-yard interval on this field as its own state, in addition to the end zones (`S`, `TD`, and `FG`).
- The drive (or, in reinforcement learning terms, episode) terminates if the ball is moved into the end zone.
- At each state, the Offense may either `run` the ball or `pass` it (always in the direction of `TD`).

The head coach argues that it is not a good idea to assume a value for the transition probability from one state to the next. She also argues that since it is not the case that the Offense will always score seven points when going into the `TD` state (they can also score six points or eight) it is better to not assume a reward function. It is also possible to start the drive from anywhere in the field. For all these reasons, she encourages the offensive coordinators to try Q-learning.

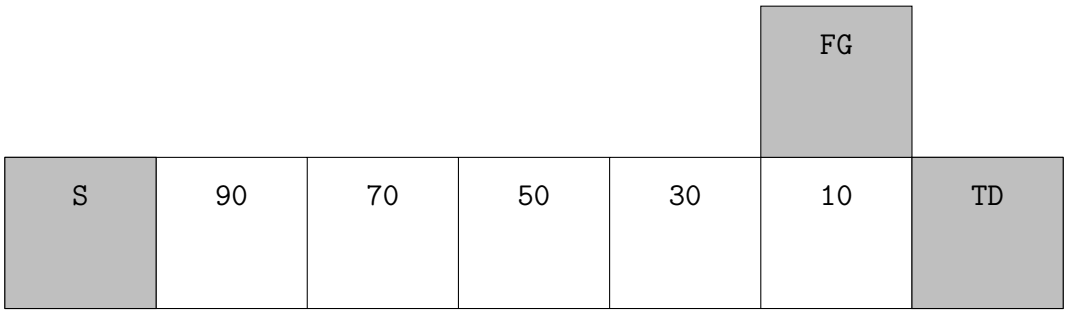
1. The offensive coordinators, now convinced, decide to build a Q-value table that matches the state and action space. What is the size of this table?

2. The offensive coordinators initialize $Q(s, a) = 0 \ \forall s, a$. They decide to update it by sampling random drives from games previously played by the Offense this season (note that this approach uses experience replay and not a standard MDP since individual plays in a football game are not independent unlike moves in a grid world). They give you the empty Q-value table for ease:

   Q($\cdot$, `run`):



Q($\cdot$, pass):



Time to watch some football :) Assume the learning rate $\alpha = 0.1$ and the discount factor $\gamma = 0.5$. Update the Q-table above for each of the following episodes using the Temporal Difference error update:

(a) **Episode 1, iteration 1**: Beginning at the 30 state, Offense runs the ball to 10. Offense collects zero reward.
$Q(30, \texttt{run}) =$

(b) **Episode 1, iteration 2**: Offense passes the ball from the 10 but stays at the 10. Offense collects zero reward.
$Q(10, \texttt{pass}) =$

(c) **Episode 1, iteration 3**: From the 10, Offense passes the ball again and reaches the TD state. The drive terminates. Offense collects a reward of 7.
$Q(10, \texttt{pass}) =$

(d) After updating the Q-function for the first episode, the head coach decides to quiz the offensive coordinators. If the Offense is in state 10, what is the best action?
$\pi(10) =$

(e) **Episode 2, iteration 1**: Beginning at the 90 state, the Offense runs the ball, causing a safety (they enter the S state). The drive terminates. Offense collects -2 reward.
$Q(90, \texttt{run}) =$

(f) After updating the Q-function for the second episode, the head coach decides to quiz the offensive coordinators again. If the Offense is in state 90, what is the

best action to take?
$\pi(\texttt{90}) =$

(g) **Episode 3, iteration 1**: Beginning at `50`, the Offense `pass`es the ball but stays at the `50`. Offense collects zero reward.
$Q(\texttt{50}, \texttt{pass}) =$

(h) **Episode 3, iteration 2**: Beginning at the `50`, the Offense `pass`es the ball to the `30`. Offense collects zero reward.
$Q(\texttt{50}, \texttt{pass}) =$

(i) **Episode 3, iteration 3**: Beginning at the `30`, the Offense `run`s the ball to the `10`. Offense collects zero reward.
$Q(\texttt{30}, \texttt{run}) =$

(j) **Episode 3, iteration 4**: Beginning at the `10`, the Offense `run`s the ball and ends the drive after entering the `FG` state. Offense collects a reward of 3 points.
$Q(\texttt{10}, \texttt{run}) =$

(k) After updating the Q-function for the third episode, the head coach decides to quiz the offensive coordinators for the last time. If the Offense is in state `10`, what is the best action to take?
$\pi(\texttt{10}) =$

4 Learning Theory

4.1 PAC Learning

1. Basic notation:
   - Probability distribution (unknown): $X \sim p^*$
   - **True function** (unknown): $c^* : X \to Y$
   - **Hypothesis space** $\mathcal{H}$ and **hypothesis** $h \in \mathcal{H} : X \to Y$
   - Training dataset $D = \{x^{(1)}, \ldots, x^{(N)}\}$
2. **True Error (expected risk)**

$$R(h) = P_{x \sim p^*(x)}(c^*(x) \neq h(x))$$

3. **Train Error (empirical risk)**

$$\begin{aligned}\hat{R}(h) &= P_{x \sim D}(c^*(x) \neq h(x)) \\ &= \frac{1}{N} \sum_{i=1}^{N} \mathbb{1}(c^*(x^{(i)}) \neq h(x^{(i)})) \\ &= \frac{1}{N} \sum_{i=1}^{N} \mathbb{1}(y^{(i)} \neq h(x^{(i)}))\end{aligned}$$

The **PAC criterion** is that we produce a high accuracy hypothesis with high probability. More formally,

$$P(\forall h \in \mathcal{H}, _____ \leq ____) \geq _____$$

Sample Complexity is the minimum number of training examples N such that the PAC criterion is satisfied for a given ϵ and δ.

Sample Complexity for 4 Cases: See Figure 6. Note that

- **Realizable** means $c^* \in \mathcal{H}$
- **Agnostic** means c^* may or may not be in $\mathcal{H}$

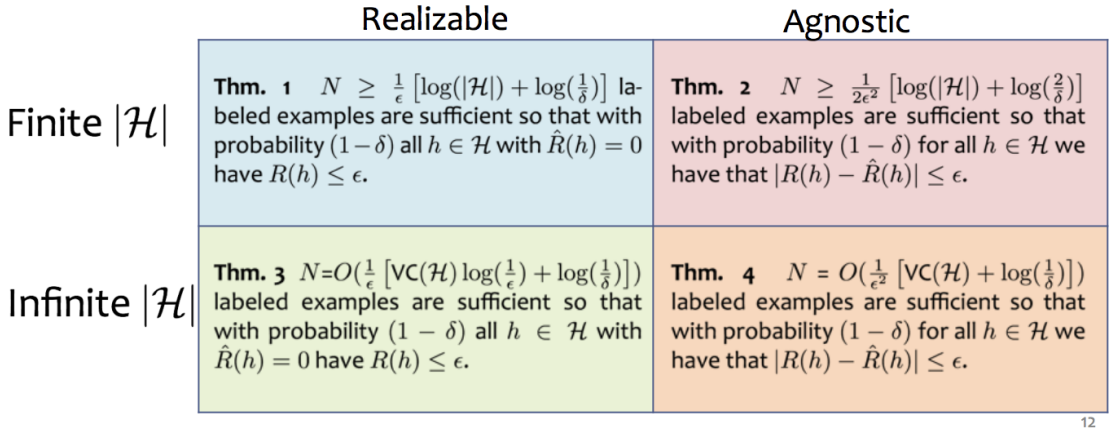


Figure 6: Sample Complexity for 4 Cases

The **VC dimension** of a hypothesis space $\mathcal{H}$, denoted $\text{VC}(\mathcal{H})$ or $d_{VC}(\mathcal{H})$, is the maximum number of points such that there exists at least one arrangement of these points and a hypothesis $h \in \mathcal{H}$ that is consistent with any labelling of this arrangement of points.

To show that $\text{VC}(\mathcal{H}) = n$:

-
-

Questions

1. For the following examples, write whether or not there exists a dataset with the given properties that can be shattered by a linear classifier.
   - 2 points in 1D
   - 3 points in 1D
   - 3 points in 2D
   - 4 points in 2D

   How many points can a linear boundary (with bias) classify exactly for d-Dimensions?

2. Consider a rectangle classifier (i.e. the classifier is uniquely defined 3 points $x_1, x_2, x_3 \in \mathbb{R}^2$ that specify 3 out of the four corners), where all points within the rectangle must equal 1 and all points outside must equal -1.

   (a) Which of the configurations of 4 points in Figure 7 can a rectangle shatter?

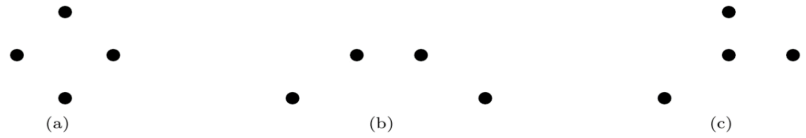


Figure 7

   (b) What about the configurations of 5 points in Figure 8?



Figure 8

3. Let $x_1, x_2, \ldots, x_n$ be n random variables that represent binary literals ($x \in \{0,1\}^n$). Let the hypothesis class $\mathcal{H}_n$ denote the conjunctions of no more than n literals in which each variable occurs at most once. Assume that $c^* \in \mathcal{H}_n$.

   Example: For $n = 4$, $(x_1 \wedge x_2 \wedge x_4), (x_1 \wedge \neg x_3) \in \mathcal{H}_4$

   Find the minimum number of examples required to learn $h \in \mathcal{H}_{10}$ which guarantees at least 99% accuracy with at least 98% confidence.